Designated-initializers for Base Classes

Document #:	P2287R5 [Latest] [Status]
Date:	2025-05-17
Project:	Programming Language C++
Audience:	CWG

Reply-to:	Barry Revzin <barry.revzin@gmail.com>

Contents

§ 1 Revision History

Since [P2287R4], wording improvements.

Since [P2287R3], clarify that the design allows you to initialize the first base class from a non-designated initializer but the second base class's members indirectly. Simplified wording as a result.

[P2287R2] proposed only allowing indirectly initializing members using the designated syntax. R3 additionally allows directly initializing base class subobjects *without* a designator. Also adds an implementation.

[P2287R1] proposed a novel way of naming base classes, as well as a way for naming indirect non-static data members. This revision *only* supports naming direct or indirect non-static data members, with no mechanism to name base classes. Basically only what Matthias suggested.

[P2287R0] proposed a single syntax for a *designated-initializer* that identifies a base class. Based on a reflector suggestion from Matthias Stearn, this revision extends the syntax to allow the brace-elision

version of *designated-initializer*: allow naming indirect non-static data members as well. Also actually correctly targeting EWG this time.

2 Introduction

[P0017R1] extended aggregates to allow an aggregate to have a base class. [P0329R4] gave us designated initializers, which allow for much more expressive and functional initialization of aggregates. However, the two do not mix: a designated initializer can currently only refer to a direct non-static data members. This means that if I have a type like:

```
struct A {
    int a;
};

struct B : A {
    int b;
};
```

While I can initialize an `A` like `A{.a=1}`, I cannot designated-initialize `B`. An attempt like `B{{.a=1}, .b=2}` runs afoul of the rule that the initializers must either be all designated or none designated. But there is currently no way to designate the base class here.

Which means that my only options for initializing a `B` are to fall-back to regular aggregate initialization and write either `B{{1}, 2}` or `B{1, 2}`. Neither are especially satisfactory.

2.1 Design Space

There are basically three potential approaches for being able to designated-initialize `B`:

```
// provide a way to name the base class
auto b1 = B{.A={.a=1}, .b=2};

// allow mixing designated and non-designated
auto b2 = B{{.a=3}, .b=4};

// allow directly initializing indirect members
auto b3 = B{.a=5, .b=6};
```

A previous revision of this paper proposed allowing only `b1` — coming up with a way to name the base class. This is much more complicated than naming an aggregate member because base classes aren't just *`identifier`*s, they can include template parameters. This revision eschews that approach entirely.

This leaves the other two options. When this paper was discussed in Varna, there was some desire expressed for having braces around the base class subobject (as in `b2` above). While allowing for optional braces is straightforward enough (and is simply a matter of refining the restriction on mixing designated and non-designated initializers), I would be strongly opposed to mandating braces.

Designated-initialization mirrors assignment. The goal is to do this substitution:

Assignment	Initialization

`[&]{`
`  B b;`
`  b.a = 5;`
`  b.b = 6;`
`  return b;`
`}();`
	`B{.a=5, .b=6}`

We can directly assign into `a`, so we should be able to directly initialize into it. Of course, in C the two forms are identical (assuming C had lambdas) while in C++ they aren't necessarily due to the existence of constructors. This makes designated initialization in C a much simpler problem. Plus C doesn't have base classes. But the essence should be the same — mandating braces breaks that.

Additionally, in many cases, the point of inheritance of aggregates is not because we actually need an "is-a" relationship but rather to compose aggregate members. I don't need `B` to be an `A`, I need `B` to have the same members as `A` with the same convenient access syntax — as opposed to having a member of type `A` and needing an extra accessor in the chain. Forcing the user to initialize `B` with braces forces the user to be aware of what is basically an implementation detail. It's frustrating enough to deal with the requirement that initializers be in-order, additionally requiring braces is too much.

Lastly, aggregate initialize *already* allows for brace elision. `B{1, 2}` is well-formed today, despite the `1` initializing a base class subobject and `2` initializing a direct member. So it strikes me as especially contrary to the design to suddenly mandate braces here.

But *allowing* (not mandating) braces? That seems totally fine. Also, notably, gcc in `-std=c++17` mode — still to this day on trunk — supports `B{{.a=1}, .b=2}`. We actually had some code break while upgrading to C++20 that initialized aggregates in this way. While I think `B{.a=1, .b=2}` is the clearest way to initialize this type, `B{{.a=1}, .b=2}` is still great — and both are substantial improvements over the best you can do in valid C++20 today, which would be either `B{{.a=1}, 2}` or simply `B{1, 2}`.

3 Proposal

This paper proposes extending designated initialization syntax to:

1. allow directly initializing base class aggregate elements, and
2. allow mixing designated and non-designated initializers, if all of the non-designated initializers are at the beginning of the *initializer-list* and are used to initialize base class subobjects.

In short, based on the above declarations of `A` and `B`, this proposal allows all of the following declarations:

```
B{{1}, 2}          // already valid in C++17
B{1, 2}            // already valid in C++17
```

```
B{.a=1, .b=2}     // proposed
B{{.a=1}, .b=2}   // proposed
B{.a{1}, .b{2}}   // proposed
B{.b=2, .a=1}     // still ill-formed
```

§ 3.1 Naming the base classes

The original revisions of this paper dealt with how to name the `A` base class of `B`, and what this means for more complicated base classes (such at those with template parameters). This revision eschews that approach entirely: it's simpler to just stick with naming members, direct and indirect. After all, that's how these aggregates will be interacted with.

What this means is that while this paper proposes that this works:

```
struct A { int a; };
struct B : A { int b; };

auto b = B{.a=1, .b=2};
```

And in a type that has a base class that is not an aggregate, you can use the mix-and-match form if the non-designated initializers initialize base classes

```
struct C : std::string { int c1, c2; };

auto c1 = C{"hello", .c1=3, .c2=4};    // ok
auto c2 = C{{"hello"}, .c1=3, .c2=4};  // ok
auto  c3  =  C{"nope",  3,  .c2=4};            // ill-formed: all the non-
        designated initializers
                                               // have to initialize a base
        class
```

If you have a hierarchy with repeated members, you'll likewise have to use the mix-and-match form:

```
struct D { int x; };
struct E : D { int x; };

auto e1 = E{.x=1};         // ok: initializes D::x to {} and E::x to 1
auto e2 = E{{.x=1}, .x=2}; // ok: initializes D::x to 1 and E::x to 2
auto e3 = E{D{1}, .x=2};   // ok: initializes D to D{1} and E::x to 2
```

If you have two base classes, they do not need to look the same. It's just that all the non-designated initializers have to be at the front:

```
struct F { int f; };
struct G { int g; };
struct H : F, G { int h; };
```

```
auto h1 = H{{.f=1}, {.g=2}, .h=3};      // ok
auto h2 = H{{.f=1}, .g=2, .h=3};         // ok, not all bases have to be
        the same
auto h3 = H{{.f=1}, G{2}, .h=3};         // ok, likewise
auto h4 = H{{.f=1} {.g=2}, .g=3, .h=4}; // ill-formed: attempting to
        initialize the G subobject in
                                         // different ways
```

Coming up with a way to *name* the base class subobject of a class seems useful, but that's largely orthogonal. It can be done later.

§ 3.2 Naming all the subobjects

The current wording we have says that, from 9.5.2 [dcl.init.aggr]/3.1:

> (3.1) If the initializer list is a brace-enclosed *designated-initializer-list*, the aggregate shall be of class type, the *identifier* in each designator shall name a direct non-static data member of the class […]

And, from 9.5.5 [dcl.init.list]/3.1 (conveniently, it's 3.1 in both cases):

> (3.1) If the *braced-init-list* contains a *designated-initializer-list*, `T` shall be an aggregate class. The ordered identifiers in the designators of the *designated-initializer-list* shall form a subsequence of the ordered identifiers in the direct non-static data members of `T`. Aggregate initialization is performed ([dcl.init.aggr]).

The proposal here is to extend both of these rules to cover not just the direct non-static data members of `T` but also all indirect members, such that every interleaving base class is also an aggregate class. That is:

```
struct A { int a; };
struct B : A { int b; };
struct C : A { C(); int c; };
struct D : C { int d; };

A{.a=1};       // okay since C++17
B{.a=1, .b=2}; // proposed okay, 'a' is a direct member of an aggregate
        class
               // and A is a direct base
C{.c=1};       // error: C is not an aggregate
D{.a=1};       // error: 'a' is a direct member of an aggregate class
               // but an intermediate base class (C) is not an aggregate
```

Or, put differently, every identifier shall name a non-static data member that is not a (direct or indirect) member of any base class that is not an aggregate.

Also this is still based on lookup rules, so if the same name appears in multiple base classes, then either it's only the most derived one that counts:

```
struct X { int x; };
struct Y : X { int x; };
Y{.x=1}; // initializes Y::x
```

or is ambiguous:

```
struct X { int x; };
struct Y { int x; };
struct Z : X, Y { };
Z{.x=1}; // error:: ambiguous which X
```

would be ill-formed on the basis that `Z::x` is ambiguous.

3.3 Impact on Existing Code

There is one case I can think of where code would change meaning:

```
struct A { int a; };
struct B : A { int b; };

void f(A); // #1
void f(B); // #2

void g() {
    f({.a=1});
}
```

In C++23, `f({.a=1})` calls `#1`, as it's the only viable candidate. But with this change, `#2` also becomes a viable candidate, so this call becomes ambiguous. I have no idea how much such code exists. This is, at least, easy to fix.

I don't think there's a case where code would change from one valid meaning to a different valid meaning - just from valid to ambiguous.

3.4 Implementation Experience

I implemented this in clang in a very literal way — by synthesizing a new designated-initializer-list to initialize the base classes in the situations where that comes up. There is probably a more straightforward way to do this, but all the examples in the paper work.

4 Wording

§ 4.1 Strategy

The wording strategy here is as follows. Let's say we have this simple case:

```
struct A { int a; };
struct B : A { int b; };
```

In the current wording, `B` has two elements (the `A` direct base class and then the `b` member). The initialization `B{.b=2}` is considered to have one explicitly initialized element (the `b`, initialized with `2`) and then the `A` is not explicitly initialized and cannot have a default member initializer, so it is copy-initialized from `{}`.

The strategy to handle `B{.a=1, .b=2}` is to group the indirect non-static data members under their corresponding direct base class and to treat those base class elements as being explicitly initialized. So here, the `A` element is explicitly initialized from `{.a=1}` and the `b` element continues to be explicitly initialized from `2`. And then this applies recursively, so given:

```
struct C : B { int c; };
```

With `C{.a=1, .c=2}`, we have:

- the `B` element is explicitly initialized from `{.a=1}`, which leads to:
  - the `A` element is explicitly initialized from `{.a=1}`, which leads to:
    - the `a` element is explicitly initialized from `1`
  - the `b` element is not explicitly initialized and has no default member initializer, so it is copy-initialized from `{}`
- the `c` element is explicitly initialized from `2`

§ 4.2 Actual Wording

Change the grammar in 9.5.1 [dcl.init.general]/1 to allow a *designated-initializer-list* to start with an *initializer-list*:

```
+ designated-only-initializer-list:
+   designated-initializer-clause
+   designated-only-initializer-list , designated-initializer-clause

  designated-initializer-list:
-   designated-initializer-clause
-   designated-initializer-list , designated-initializer-clause
+   designated-only-initializer-list
+   initializer-list , designated-only-initializer-list
```

Add a new term after we define what an aggregate and the elements of an aggregate are in 9.5.2 [dcl.init.aggr] and then extend the next sections.

1 An *aggregate* is […]

2 The *elements* of an aggregate are: […]

x The *associated element* of a member `M` of an aggregate `T` is:

(x.1) — `M` if `M` is an element of `T`;

(x.2) — otherwise, the element of `T` that contains `M`.

[*Example 1:*

```
struct A {
  int a1;
  union {
    int a2;
    char a3;
  };
};

struct B : A {
  int b1;
  union {
    double b2;
  };
};
```

The associated element of each of the members `A::a1`, `A::a2`, and `A::a3` of `B` is `A`. The associated element of the member `B::b1` of `B` is itself. The associated element of the member `B::b2` of `B` is the anonymous union containing it. — *end example*]

3 When an aggregate is initialized by an initializer list as specified in [dcl.init.list], the elements of the initializer list are taken as initializers for the elements of the aggregate. The *explicitly initialized elements* of the aggregate are determined as follows:

(3.1) — ~~If the initializer list is a brace-enclosed *designated-initializer-list*~~ If the *braced-init-list* has a *designated-only-initializer-list* within a *designated-initializer-list*, the aggregate shall be of class type *C*. ~~, the identifier in each *designator* shall name a direct non-static data member of the class, and the explicitly initialized elements of the aggregate are the elements that are, or contain, those members.~~

For each *designator*, the lookup set for the *identifier* in *C* ([class.member.lookup]) shall comprise

(3.1.1) — a declaration set consisting of a single non-static data member and

(3.1.2) — a subobject set containing only one subobject, whose type shall be an aggregate base of *C*. An aggregate *B* is an *aggregate base* of a class *D* if it is *D* or a direct

base class of an aggregate base of *D*.

That non-static data member is *designated* by the *identifier*. Each *initializer-clause*, if any, shall appertain (see below) to a base class subobject of *C*. The explicitly initialized elements of the aggregate include the associated elements of each member *M* of *C* for which *M* is designated by an *identifier* in a *designated-initializer-clause*.

(3.2) — ~~If the initializer list is a brace-enclosed *initializer-list*~~ If the *braced-init-list* has an *initializer-list* (possibly within a *designated-initializer-list*), the explicitly initialized elements of the aggregate ~~are~~ include those for which an element of the initializer list appertains to the aggregate element or to a subobject thereof (see below).

(3.3) — ~~Otherwise, the initializer list must be {}, and there are no explicitly initialized elements.~~ No other elements of the aggregate are explicitly initialized. [*Note 1:* The initializer `{}` does not explicitly initialize any elements of the aggregate. — *end note*]

If any element of the aggregate is explicitly initialized by both an *initializer-list* and a *designated-initializer-list*, the program is ill-formed. If a non-static data member is explicitly initialized by an *initializer-list*, a *designated-only-initializer-list* shall not be present.

[*Example 2:*

```
struct A { int a1, a2; };
struct B : A { int b; };
struct C : A { int a1; };

B v1 = B{.a1=1, .b=2};          // the explicitly initialized elements
       are [A, B::b]
B v2 = B{.a1=1, .a2=2, .b=3};   // the explicitly initialized elements
       are [A, B::b]
B v3 = B{A{1, 2}, .b=3};        // the explicitly initialized elements
       are [A, B::b]
B v4 = B{A{}, .a2=1, .b=3};     // error: A initialized two different
       ways
C v5 = C{.a1=4};                // the explicitly initialized elements
       are [C::a1]
```

— *end example*]

4 For each explicitly initialized element:

(4.1) — If the element is an anonymous union member and ~~the initializer list is a brace-enclosed *designated-initializer-list*~~ is explicitly initialized by a *designated-only-initializer-list*, the element is initialized by the *braced-init-list* `{ D }`, where *D* is the *designated-initializer-clause* whose associated element is the anonymous union member ~~naming a member of the anonymous union member~~. There shall be only one such *designated-initializer-clause*.

(4.2) — Otherwise, if the ~~initializer list is a brace-enclosed *designated-initializer-list*~~ element is explicitly initialized by a *designated-only-initializer-list*, then

(4.2.1) — if the element is a direct, non-static data member, then the element is initialized with the *brace-or-equal-initializer* of the corresponding *designated-initializer-clause*. If that initializer is of the form `=` *assignment-expression* and a narrowing conversion ([dcl.init.list]) is required to convert the expression, the program is ill-formed. [*Note 2:* The form of the initializer determines whether copy-initialization or direct-initialization is performed. — *end note*]

(4.2.2) — otherwise, the element is a base class subobject *B*, and is copy-initialized from a brace-enclosed *designated-initializer-list* consisting of all of the *designated-initializer-clause*s whose associated element is *B*, in order.

[*Example 3:*

```
struct A { int a; };
struct B : A { int b; };
struct C : B { int c; };

// the A element is initialized from {.a=1}
B x = B{.a=1};

// the B element is initialized from {.a=2, .b=3}
// which leads to its A element being initialized from {.a=2}
C y = C{.a=2, .b=3, .c=4};
```

— *end example*]

(4.3) — Otherwise, the ~~initializer list is a brace-enclosed *initializer-list*~~ element is explicitly initialized by an *initializer-list*. […]

Extend 9.5.5 [dcl.init.list]/3.1:

(3.1) — If the *braced-init-list* contains a *designated-initializer-list*, `T` shall be an aggregate class. The associated elements of the non-static data members designated by the ordered *identifier*s in the designators of the *designated-initializer-list* shall ~~form a subsequence of the ordered *identifiers* in the direct non-static data members~~ be in non-decreasing declaration order. Aggregate initialization is performed ([dcl.init.aggr]).

[*Example 4:*

```
struct A { int x; int y; int z; };
A a{.y = 2, .x = 1};                // error: designator order
    does not match declaration order
A b{.x = 1, .z = 2};                // OK, b.y initialized to 0

+   struct B : A { int q; };
```

```
+    B e{.x = 1, .q = 3};                    // OK, e.y and e.z
         initialized to 0
+   B f{.q = 3, .x = 1};                  // error: designator order
         does not match declaration order

+   struct C { int p; int x; };
+   struct D : A, C { };
+   D g{.y=1, .p=2};                      // OK
+   D h{.x=2};                             // error: ambiguous lookup
         for x

+   struct NonAggr { int na; NonAggr(int); };
+   struct E : NonAggr { int e; };
+    E i{.na=1, .e=2};                      // error: the lookup set
         for na finds NonAggr, which is not an aggregate base of E
```

— end example]

And update paragraph 14, since now we can have *`initializer-clause`*s in *`designated-initializer-list`*s too:

> 14 Each *`initializer-clause`* in a ~~brace-enclosed *`initializer-list`*~~ *`braced-init-list`* is said to *appertain* to an element of the aggregate being initialized or to an element of one of its subaggregates.

Add an Annex C entry:

Affected sublcause: [dcl.init]
Change: Support for designated initialization of base classes of aggregates.
Rationale: New functionality.
Effect on original feature: Some valid C++23 code may fail to compile. For example:

```
struct A { int a; };
struct B : A { int b; };

void f(A); // #1
void f(B); // #2

void g() {
    f({.a=1}); // ambiguous between #1 and #2; previously called #1

}
```

§ 4.3 Feature-test Macro

Bump `__cpp_designated_initializers` in 15.12 [cpp.predefined]:

```
- __cpp_designated_initializers 201707L
```

```
+ __cpp_designated_initializers 2025XXL
```

5 Acknowledgements

Thanks to Matthias Stearn for, basically, the proposal. Thanks to Tim Song for helping with design questions and wording. Thanks to Brian Bi and Davis Herring for a lot of help with the wording.

6 References

[P0017R1] Oleg Smolsky. 2015-10-24. Extension to aggregate initialization.
https://wg21.link/p0017r1

[P0329R4] Tim Shen, Richard Smith. 2017-07-12. Designated Initialization Wording.
https://wg21.link/p0329r4

[P2287R0] Barry Revzin. 2021-01-21. Designated-initializers for base classes.
https://wg21.link/p2287r0

[P2287R1] Barry Revzin. 2021-02-15. Designated-initializers for base classes.
https://wg21.link/p2287r1

[P2287R2] Barry Revzin. 2023-03-12. Designated-initializers for base classes.
https://wg21.link/p2287r2

[P2287R3] Barry Revzin. 2024-09-10. Designated-initializers for base classes.
https://wg21.link/p2287r3

[P2287R4] Barry Revzin. 2025-03-11. Designated-initializers for base classes.
https://wg21.link/p2287r4